

Clinician Portal - Main Portal & Role-Based Access Requirements

Overview

The Clinician Portal serves as a role-based dashboard that provides access to specialized interfaces tailored to different clinical and administrative roles within the medical practice. The portal implements strict role-based access control (RBAC) and provides intuitive navigation to role-specific dashboards.

1. Portal Architecture & Layout

1.1 Main Portal Landing Page

Visual Requirements:

- **Page Title:** "Clinician Portal" (32px, bold, center-aligned)
- **Subtitle:** "Access specialized dashboards tailored to your role. Please select your role to continue." (16px, gray, center-aligned)
- **Background:** Light gradient with subtle medical iconography
- **Card Layout:** 2x3 grid on desktop, single column on mobile
- **Card Spacing:** 24px gaps between cards
- **Overall Padding:** 48px on desktop, 24px on mobile

Header Navigation:

```
typescript
interface ClinicianPortalHeader {
  title: "Clinician Portal";
  showBackButton: boolean; // True if user came from another portal
  showUserProfile: boolean; // Show current user info and logout
  breadcrumb?: string; // e.g., "Patient Portal > Clinician Portal"
}
```

1.2 Role Selection Cards

Card Design Specifications:

- **Card Size:** 300px width × 200px height on desktop
- **Border Radius:** 12px

- **Shadow:** 0 4px 6px rgba(0, 0, 0, 0.1)
 - **Hover Effect:** Lift with increased shadow (0 8px 15px rgba(0, 0, 0, 0.15))
 - **Background:** White with subtle icon watermark
 - **Transition:** All hover effects 200ms ease-in-out
-

2. Role-Based Access Control (RBAC)

2.1 Role Definitions & Permissions

Core Role Structure:

```
typescript

interface ClinicalRole {
  id: string;
  name: string;
  description: string;
  icon: string;
  permissions: Permission[];
  dashboardRoute: string;
  accessLevel: 'basic' | 'advanced' | 'admin';
  requiresAdditionalAuth?: boolean;
}

interface Permission {
  resource: string; // 'patients' | 'appointments' | 'billing' | 'lab_results'
  actions: ('read' | 'write' | 'delete' | 'approve')[];
  constraints?: {
    ownPatientsOnly?: boolean;
    timeRestricted?: boolean;
    requiresSupervision?: boolean;
  };
};
```

Role Definitions:

1. Doctor Role:

```
typescript
```

```
const doctorRole: ClinicalRole = {
  id: "doctor",
  name: "Doctor",
  description: "Access patient charts, manage appointments, review lab results, and handle secure messaging.",
  icon: "🩺", // Stethoscope icon
  dashboardRoute: "/clinician/doctor",
  accessLevel: "admin",
  permissions: [
    {
      resource: "patients",
      actions: ["read", "write", "approve"],
      constraints: { ownPatientsOnly: false }
    },
    {
      resource: "prescriptions",
      actions: ["read", "write", "approve"]
    },
    {
      resource: "lab_results",
      actions: ["read", "write", "approve"]
    },
    {
      resource: "diagnoses",
      actions: ["read", "write", "approve"]
    },
    {
      resource: "billing_codes",
      actions: ["read", "write", "approve"]
    }
  ]
};
```

2. Medical Assistant (MA) Role:

typescript

```
const medicalAssistantRole: ClinicalRole = {
  id: "medical_assistant",
  name: "Medical Assistant",
  description: "View the patient queue, manage room assignments, and record patient vitals before the doctor's visit.",
  icon: "🩺", // Medical icon
  dashboardRoute: "/clinician/ma",
  accessLevel: "advanced",
  permissions: [
    {
      resource: "patients",
      actions: ["read", "write"],
      constraints: { ownPatientsOnly: true }
    },
    {
      resource: "vitals",
      actions: ["read", "write"]
    },
    {
      resource: "room_management",
      actions: ["read", "write"]
    },
    {
      resource: "patient_intake",
      actions: ["read", "write"]
    }
  ]
};
```

3. Technician Role:

typescript

```
const technicianRole: ClinicalRole = {
  id: "technician",
  name: "Technician",
  description: "View assigned lab orders, enter results, and manage laboratory equipment and schedules.",
  icon: "🧪", // Test tube icon
  dashboardRoute: "/clinician/technician",
  accessLevel: "basic",
  permissions: [
    {
      resource: "lab_orders",
      actions: ["read", "write"]
    },
    {
      resource: "lab_results",
      actions: ["read", "write"],
      constraints: { requiresSupervision: true }
    },
    {
      resource: "equipment",
      actions: ["read", "write"]
    }
  ]
};
```

4. Back Office Role:

typescript

```
const backOfficeRole: ClinicalRole = {
  id: "back_office",
  name: "Back Office",
  description: "Manage patient registration, track lab results, and handle billing inquiries.",
  icon: "📋", // Clipboard icon
  dashboardRoute: "/clinician/back-office",
  accessLevel: "advanced",
  permissions: [
    {
      resource: "patient_registration",
      actions: ["read", "write", "approve"]
    },
    {
      resource: "insurance_verification",
      actions: ["read", "write"]
    },
    {
      resource: "billing_inquiries",
      actions: ["read", "write"]
    },
    {
      resource: "lab_tracking",
      actions: ["read", "write"]
    }
  ]
};
```

5. Front Office Role:

typescript

```
const frontOfficeRole: ClinicalRole = {
  id: "front_office",
  name: "Front Office",
  description: "Manage provider schedules, patient check-ins, and the intelligent waitlist.",
  icon: "📅", // Calendar icon
  dashboardRoute: "/clinician/front-office",
  accessLevel: "basic",
  permissions: [
    {
      resource: "appointments",
      actions: ["read", "write", "delete"]
    },
    {
      resource: "patient_checkin",
      actions: ["read", "write"]
    },
    {
      resource: "scheduling",
      actions: ["read", "write"]
    },
    {
      resource: "waitlist_management",
      actions: ["read", "write"]
    }
  ]
};
```

2.2 Role Detection & Validation

User Role Assignment:

typescript

```

interface UserSession {
    userId: string;
    assignedRoles: string[]; // User can have multiple roles
    primaryRole: string; // Default role for portal entry
    lastAccessedRole: string; // Remember last selected role
    rolePermissions: Map<string, Permission[]>;
    sessionExpiry: string;
    facilityId: string; // Multi-facility support
}

// Role validation logic
class RoleValidator {
    static canAccessRole(userSession: UserSession, requestedRole: string): boolean {
        return userSession.assignedRoles.includes(requestedRole);
    }

    static canPerformAction(
        userSession: UserSession,
        resource: string,
        action: string,
        context?: any
    ): boolean {
        const permissions = userSession.rolePermissions.get(userSession.primaryRole);
        const resourcePermission = permissions?.find(p => p.resource === resource);

        if (!resourcePermission || !resourcePermission.actions.includes(action as any)) {
            return false;
        }

        // Check constraints
        if (resourcePermission.constraints) {
            return this.validateConstraints(resourcePermission.constraints, context);
        }

        return true;
    }
}

```

3. Role Card Implementation

3.1 Card Component Specifications

Individual Role Cards:

Doctor Card:

typescript

```
interface DoctorCard {  
  title: "Doctor";  
  icon: "👨‍⚕️";  
  iconColor: "#2563eb"; // Blue  
  description: "Access patient charts, manage appointments, review lab results, and handle secure messaging.";   
  buttonText: "Go to Dashboard";  
  buttonColor: "#2563eb";  
  route: "/clinician/doctor";  
  accessLevel: "High";  
  features: [  
    "Patient Chart Access",  
    "Prescription Management",  
    "Lab Result Review",  
    "Clinical Documentation"  
  ];  
}
```

Medical Assistant Card:

typescript

```
interface MedicalAssistantCard {  
  title: "Medical Assistant";  
  icon: "👩‍⚕️";  
  iconColor: "#059669"; // Green  
  description: "View the patient queue, manage room assignments, and record patient vitals before the doctor's visit.";   
  buttonText: "Go to Dashboard";  
  buttonColor: "#059669";  
  route: "/clinician/ma";  
  accessLevel: "Medium";  
  features: [  
    "Patient Queue Management",  
    "Room Assignments",  
    "Vital Signs Entry",  
    "Pre-visit Preparation"  
  ];  
}
```

Technician Card:

typescript

```
interface TechnicianCard {  
  title: "Technician";  
  icon: "🧪";  
  iconColor: "#7c3aed"; // Purple  
  description: "View assigned lab orders, enter results, and manage laboratory equipment and schedules.";  
  buttonText: "Go to Dashboard";  
  buttonColor: "#7c3aed";  
  route: "/clinician/technician";  
  accessLevel: "Medium";  
  features: [  
    "Lab Order Processing",  
    "Result Entry",  
    "Equipment Management",  
    "Quality Control"  
  ];  
}
```

Back Office Card:

typescript

```
interface BackOfficeCard {  
  title: "Back Office";  
  icon: "📋";  
  iconColor: "#dc2626"; // Red  
  description: "Manage patient registration, track lab results, and handle billing inquiries.";  
  buttonText: "Go to Dashboard";  
  buttonColor: "#dc2626";  
  route: "/clinician/back-office";  
  accessLevel: "Medium";  
  features: [  
    "Patient Registration",  
    "Insurance Verification",  
    "Billing Support",  
    "Administrative Tasks"  
  ];  
}
```

Front Office Card:

typescript

```
interface FrontOfficeCard {  
  title: "Front Office";  
  icon: "📅";  
  iconColor: "#ea580c"; // Orange  
  description: "Manage provider schedules, patient check-ins, and the intelligent waitlist.";  
  buttonText: "Go to Dashboard";  
  buttonColor: "#ea580c";  
  route: "/clinician/front-office";  
  accessLevel: "Medium";  
  features: [  
    "Appointment Scheduling",  
    "Patient Check-in",  
    "Waitlist Management",  
    "Reception Duties"  
  ];  
}
```

3.2 Card Interaction Behavior

Click Handler Logic:

typescript

```

const handleRoleSelection = async (selectedRole: ClinicalRole) => {
  // 1. Validate user has access to this role
  if (!RoleValidator.canAccessRole(userSession, selectedRole.id)) {
    showErrorNotification("You don't have access to this role. Please contact your administrator.");
    return;
  }

  // 2. Check if additional authentication is required
  if (selectedRole.requiresAdditionalAuth) {
    const authResult = await requestAdditionalAuth(selectedRole.id);
    if (!authResult.success) {
      showErrorNotification("Additional authentication failed.");
      return;
    }
  }

  // 3. Update user session with selected role
  await updateUserSession({
    primaryRole: selectedRole.id,
    lastAccessedRole: selectedRole.id,
    roleTimestamp: new Date().toISOString()
  });

  // 4. Log role access for audit trail
  auditLog.info('Role access', {
    userId: userSession.userId,
    role: selectedRole.id,
    timestamp: new Date().toISOString(),
    ipAddress: getClientIP()
  });

  // 5. Navigate to role-specific dashboard
  router.push(selectedRole.dashboardRoute);
};

```

Accessibility Implementation:

tsx

```

const RoleCard: React.FC<{ role: ClinicalRole }> = ({ role }) => {
  return (
    <div
      className="role-card"
      role="button"
      tabIndex={0}
      aria-label={`Access ${role.name} dashboard`}
      onKeyDown={(e) => {
        if (e.key === 'Enter' || e.key === ' ') {
          handleRoleSelection(role);
        }
      }}
    >
      <div className="role-icon" aria-hidden="true">{role.icon}</div>
      <h3>{role.name}</h3>
      <p>{role.description}</p>
      <button
        onClick={() => handleRoleSelection(role)}
        aria-describedby={` ${role.id}-description`}
      >
        Go to Dashboard
      </button>
    </div>
  );
};

```

4. Responsive Design & Layout

4.1 Desktop Layout (≥ 1024px)

- **Grid:** 2 columns × 3 rows
- **Card Size:** 300px × 200px
- **Container Max Width:** 1200px
- **Centering:** Horizontal center with equal margins

4.2 Tablet Layout (768px-1023px)

- **Grid:** 2 columns × 3 rows (smaller cards)
- **Card Size:** 280px × 180px
- **Container Padding:** 32px

- **Gap:** 20px between cards

4.3 Mobile Layout ($\leq 767\text{px}$)

- **Layout:** Single column, stacked vertically
- **Card Size:** Full width with 16px margins
- **Card Height:** 120px (more compact)
- **Vertical Spacing:** 16px between cards

4.4 Responsive Card Content:

CSS

```
/* Desktop */
```

```
@media (min-width: 1024px) {
```

```
.role-card {  
  width: 300px;  
  height: 200px;  
  padding: 24px;  
}
```

```
.role-card h3 {  
  font-size: 20px;  
  margin-bottom: 8px;  
}
```

```
.role-card p {  
  font-size: 14px;  
  line-height: 1.4;  
}  
}
```

```
/* Mobile */
```

```
@media (max-width: 767px) {
```

```
.role-card {  
  width: calc(100% - 32px);  
  height: 120px;  
  padding: 16px;  
  margin: 0 16px;  
}
```

```
.role-card h3 {  
  font-size: 18px;  
  margin-bottom: 4px;  
}
```

```
.role-card p {  
  font-size: 13px;  
  line-height: 1.3;  
  display: -webkit-box;  
  -webkit-line-clamp: 2;  
  -webkit-box-orient: vertical;  
  overflow: hidden;  
}  
}
```

5. Security & Compliance

5.1 Role-Based Security

Session Management:

```
typescript

interface SecureRoleSession {
  roleSessionId: string;
  userId: string;
  selectedRole: string;
  sessionStartTime: string;
  lastActivity: string;
  ipAddress: string;
  deviceFingerprint: string;
  permissions: Permission[];
  maxSessionDuration: number; // Role-specific session limits
}
```

Security Validations:

1. **Role Assignment Verification:** Validate user has been assigned the requested role
2. **Time-Based Access:** Some roles may have time restrictions (e.g., technician only during lab hours)
3. **Device Restrictions:** High-privilege roles may require registered devices
4. **Multi-Factor Authentication:** Doctor and back office roles may require additional auth
5. **Audit Logging:** All role access attempts logged for compliance

5.2 HIPAA Compliance

Access Logging:

```
typescript
```



```
interface RoleAccessLog {  
  timestamp: string;  
  userId: string;  
  userName: string;  
  role: string;  
  action: 'access_granted' | 'access_denied' | 'session_expired';  
  ipAddress: string;  
  userAgent: string;  
  facility: string;  
  reasonForAccess?: string; // Required for high-privilege roles  
}
```

6. Performance Requirements

6.1 Loading Standards

- **Portal Load Time:** <500ms
- **Role Validation:** <100ms
- **Navigation to Dashboard:** <300ms
- **Role Permission Check:** <50ms

6.2 Caching Strategy

- **Role Permissions:** Cache for session duration
- **User Role Assignments:** Cache with 5-minute TTL
- **Role Card Content:** Static content, cache indefinitely
- **Dashboard Routes:** Preload critical routes for faster navigation

7. Testing Requirements

7.1 Role-Based Testing

```
typescript
```

```
interface RoleTestingScenarios {  
  accessControl: [  
    "User with Doctor role can access Doctor dashboard",  
    "User without MA role cannot access MA dashboard",  
    "Multi-role user can switch between authorized roles",  
    "Unauthorized role access shows appropriate error"  
  ];  
  
  permissions: [  
    "Role permissions properly restrict feature access",  
    "Permission constraints properly enforced",  
    "Cross-role permission boundaries maintained"  
  ];  
  
  security: [  
    "Role switching requires proper authentication",  
    "Session timeout properly enforced per role",  
    "Audit logging captures all role access events"  
  ];  
}
```

7.2 Acceptance Criteria

- Each role card must display correct information and navigate to proper dashboard
- Users must only see roles they're authorized to access
- Unauthorized access attempts must be blocked and logged
- Role switching must update session and permissions correctly
- All role access must be audited for compliance
- Portal must be fully accessible via keyboard navigation
- Mobile layout must be usable on smallest supported screen size (320px)

This main portal document establishes the foundation for role-based access. Now we can create detailed individual documents for each role's specific dashboard (Doctor, MA, Technician, Back Office, Front Office) with their unique workflows and features.